

Flujo

Abner Vidal



Contenidos

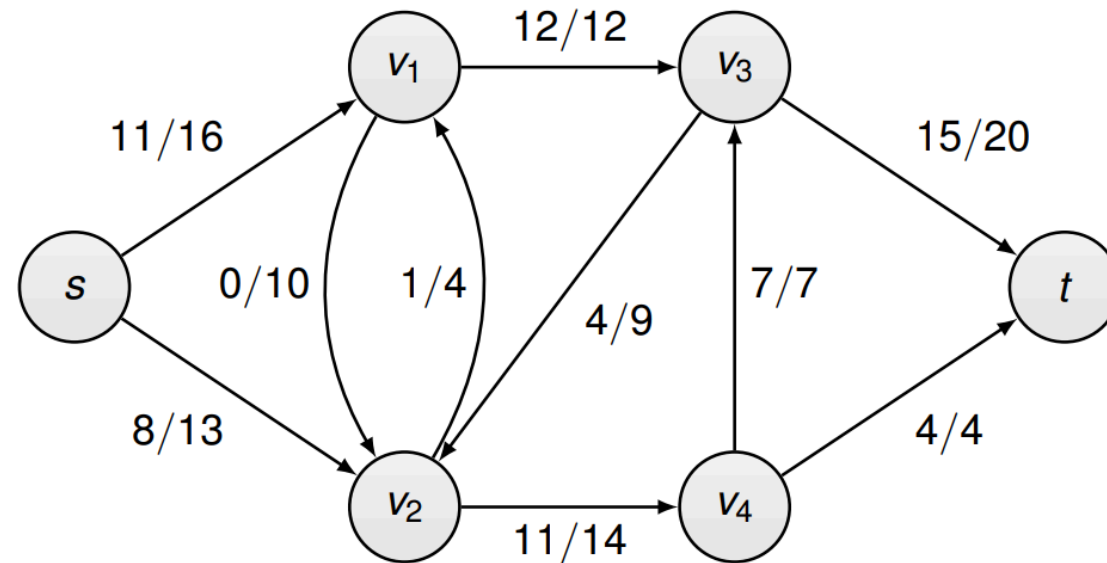
- ¿Que es flujo?
- Algoritmos para resolver flujo máximo
- Minimum cut.
- Matching bipartito.
- Minimum Vertex Cover.
- Maximum Independent Set (en grafos bipartitos).
- Maximum Closure Problem.
- Min-Cost Max-Flow
- Teorema de Dilworth



¿Que es flujo?

Imaginemos una red de cañerías por donde circula agua. En esta red existen uno o varios puntos de origen (fuentes) desde los cuales sale agua con un caudal definido, y también un punto de destino hacia donde queremos que ese agua llegue.

Cada cañería de la red tiene una capacidad máxima: un límite de caudal que puede soportar sin importar cuánta agua intente pasar por ella.





Problema

La pregunta que nos interesa responder es la siguiente: dado este sistema de cañerías con sus respectivas capacidades, ¿cuál es la máxima cantidad de agua que podemos lograr que llegue al punto de destino?

A este problema se le llama flujo máximo, y tiene varias aplicaciones:

- **Redes de telecomunicaciones:** determinar el ancho de banda máximo que se puede enviar entre dos puntos de una red, donde cada enlace tiene una capacidad limitada.
- **Emparejamiento bipartito:** asignar trabajadores a tareas, o estudiantes a universidades, modelando el problema como una red de flujo.
- Etc.

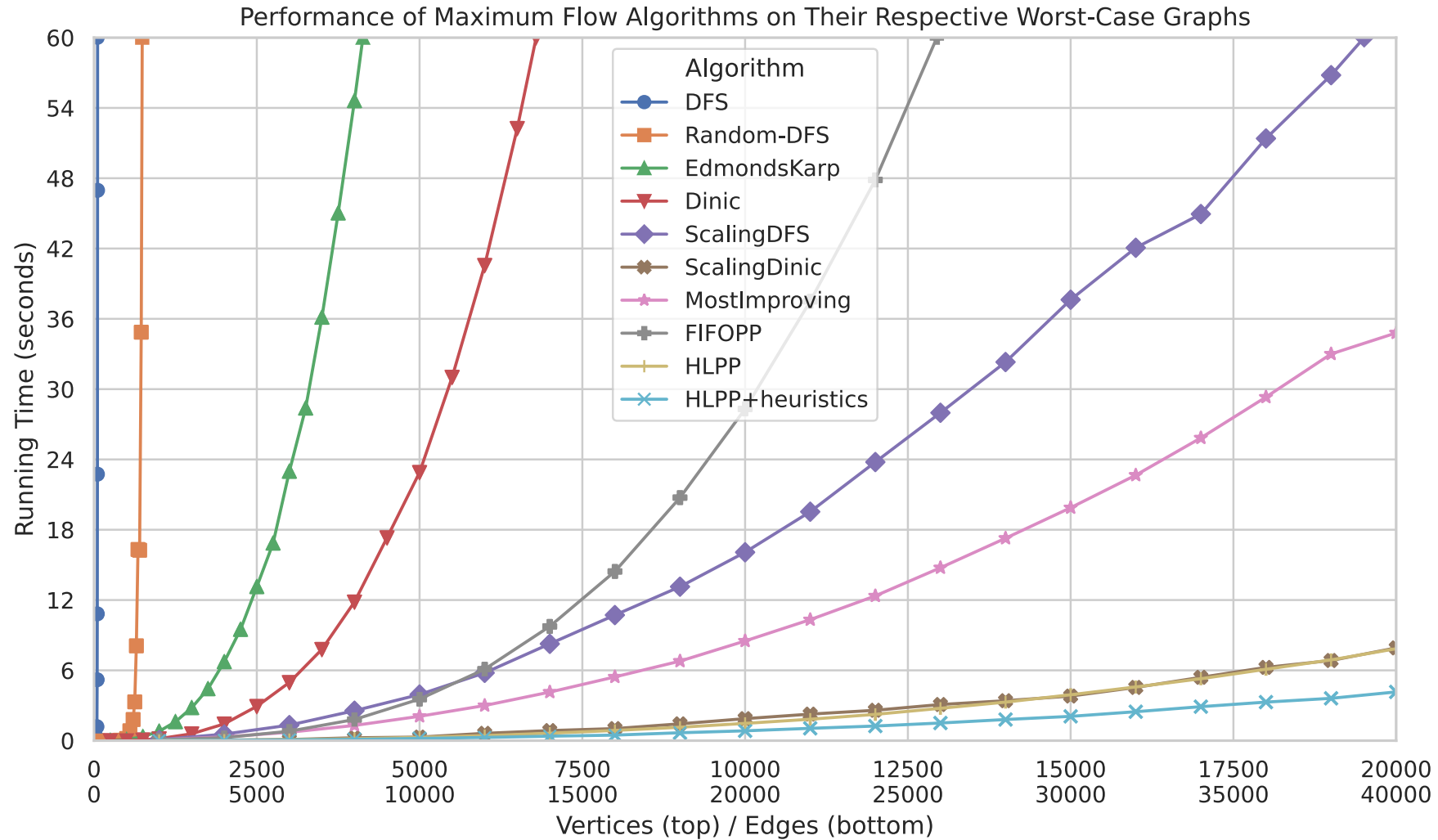


Algoritmos para resolver flujo máximo

Algoritmo	Complejidad temporal
Ford-Fulkerson con DFS	$\mathcal{O}(E^2 \cdot U)$
Edmonds-Karp	$\mathcal{O}(V \cdot E^2)$
Dinic	$\mathcal{O}(V^2 \cdot E)$
DFS con scaling	$\mathcal{O}(E^2 \cdot \log(U))$
Dinic con scaling	$\mathcal{O}(V \cdot E \cdot \log(U))$
Most improving augmenting paths	$\mathcal{O}(E \cdot \log(U) \cdot (E + V \cdot \log(V)))$
FIFO preflow push	$\mathcal{O}(V^3)$
Highest-label preflow push	$\mathcal{O}(V^2 \cdot \sqrt{E})$



Comparación de performance





¿Qué algoritmo escoger?

En programación competitiva, cuando se trata de problemas de flujo, el algoritmo que generalmente se prefiere es **Dinic**. Esto se debe a que, en la práctica, rara vez se alcanza su cota teórica de $O(V^2E)$: en la mayoría de los casos el tiempo de ejecución real es mucho menor, además de que es uno de los más sencillos de implementar.

Generalmente en las competencias, se permite llevar apuntes, por lo que conocer el funcionamiento interno del algoritmo no es necesario dominarlo a fondo, ya que rara vez se le hacen modificaciones.

Lo más importante es saber **modelar correctamente la red** según cada situación: definir bien los nodos, las aristas y las capacidades, para luego aplicar el algoritmo como una herramienta ya resuelta.



Algoritmo de Dinic

```
struct Dinic {
    struct Edge { int to, rev; ll f, c; };
    int n, t_; vector<vector<Edge>> G;
    vector<ll> D;
    vector<int> q, W;
    bool bfs(int s, int t) {
        W.assign(n, 0); D.assign(n, -1); D[s] = 0;
        int f = 0, l = 0; q[l++] = s;
        while (f < l) {
            int u = q[f++];
            for (const Edge &e : G[u]) if (D[e.to] == -1 && e.f < e.c)
                D[e.to] = D[u] + 1, q[l++] = e.to;
        }
        return D[t] != -1;
    }
    ll dfs(int u, ll f) {
        if (u == t_) return f;
        for (int &i = W[u]; i < (int)G[u].size(); ++i) {
            Edge &e = G[u][i]; int v = e.to;
            if (e.c <= e.f || D[v] != D[u] + 1) continue;
            ll df = dfs(v, min(f, e.c - e.f));
            if (df > 0) { e.f += df, G[v][e.rev].f -= df; return df; }
        }
        return 0;
    }
    Dinic(int N) : n(N), G(N), D(N), q(N) {}
    void add_edge(int u, int v, ll cap) {
        G[u].push_back({v, (int)G[v].size(), 0, cap});
        G[v].push_back({u, (int)G[u].size() - 1, 0, 0}); // Use cap instead of 0 if bidirectional
    }
    ll max_flow(int s, int t) {
        t_ = t; ll ans = 0;
        while (bfs(s, t)) while (ll dl = dfs(s, LLONG_MAX)) ans += dl;
        return ans;
    }
};
```




Importancia de flujo en ICPC

- En casi todas las regionales ICPC hay un problema
- La solución siempre suele ser armar el grafo y correr un algoritmo de flujo (que veremos ahora qué posibilidades hay)
- Muchos no saben/manejan flujo
- El problema de flujo no parece que sea de flujo
- Junto con geometría computacional, el equipo que tenga a alguien que se maneje especialmente bien en alguno de estos temas, puede llegar a garantizar la victoria sobre los demás equipos.



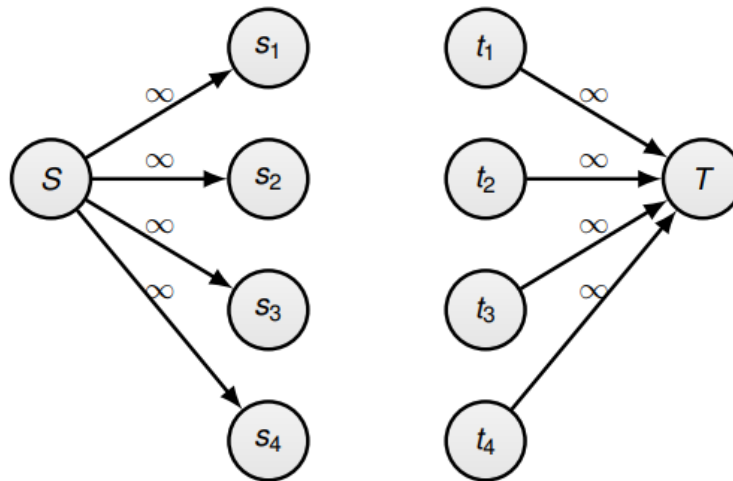
Como mencionamos antes, la habilidad clave no está en el algoritmo, sino en saber **modelar la red** según cada problema. A continuación, veremos algunos trucos y aplicaciones frecuentes a la hora de plantear ese modelado.



Múltiples fuentes/sumideros

Supongamos que tenemos un problema donde hay varios nodos de donde "sale" flujo y/o varios nodos a los que tiene que llegar flujo, ¿Qué hacemos?

- Creamos dos nodos s y t nuevos.
- Conectamos s a todos los nodos de los que "sale" flujo con capacidad ∞
- Conectamos todos los nodos a los que llega flujo con t con capacidad ∞ .

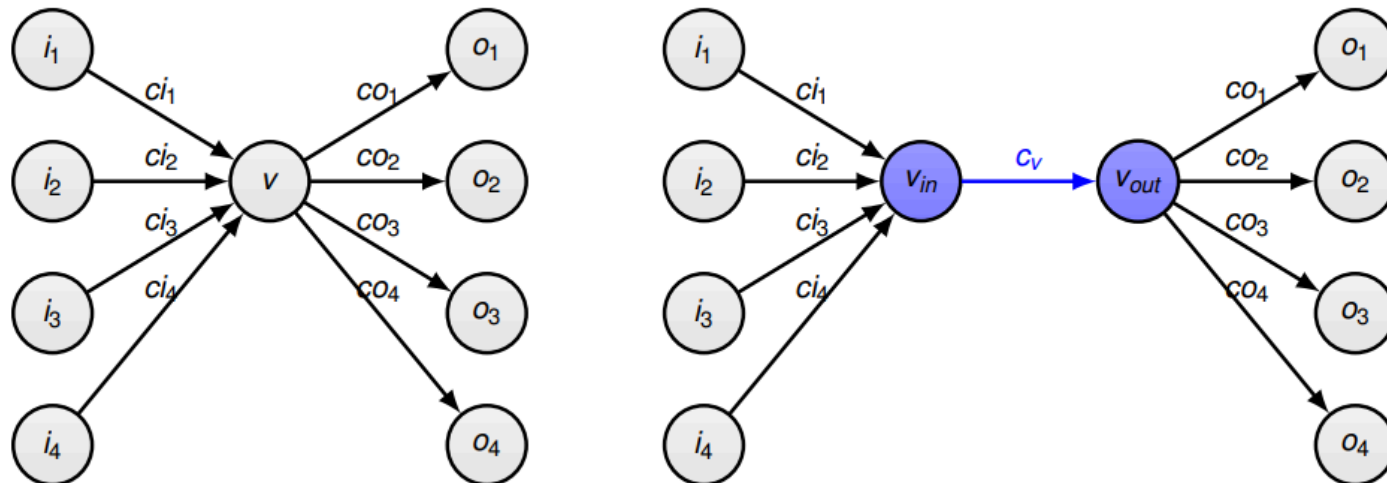




Nodos con capacidad

¿Qué pasa si tenemos restricciones de capacidad que pasa por un nodo v en lugar de por un eje?

- Duplicamos el nodo en dos nodos v_{in} y v_{out} .
- Conectamos todas las aristas que entraban a v a v_{in} .
- Conectamos todas las aristas que salían de v a v_{out} .
- Conectamos v_{in} a v_{out} con un eje de la capacidad máxima del nodo





Caso especial: grafo bipartito

En caso de que nos encontremos con un grafo bipartito, la complejidad del algoritmo de Dinic mejora considerablemente: en lugar de $O(V^2 E)$, se logra $O(E\sqrt{V})$.

Además, si todas las capacidades de las aristas son 1 (como suele ocurrir en problemas de *matching*), existe un algoritmo especializado llamado **Hopcroft-Karp**, que también logra una complejidad de $O(E\sqrt{V})$ y está diseñado específicamente para resolver el problema de emparejamiento máximo en grafos bipartitos.



El flujo puede resolver múltiples problemas, incluso problemas que intuitivamente no pareciera que estuvieran relacionados, en particular, dentro de esta presentación veremos las siguientes aplicaciones de encontrar el flujo máximo:

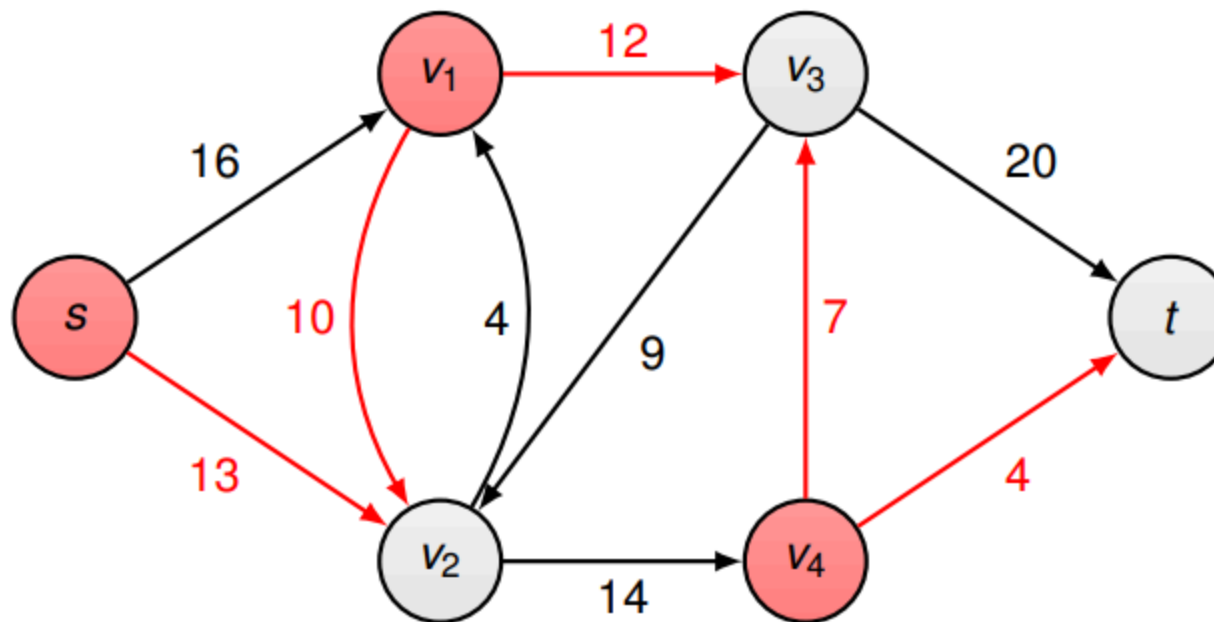
- Minimum cut.
- Matching bipartito.
- Minimum Vertex Cover.
- Maximum Indenpendent Set (en grafos bipartitos).
- Maximum Closure Problem.



Definición de corte

Un corte en una red es una partición de los nodos en dos conjuntos U y V tal que $s \in U$ y $t \in V$.

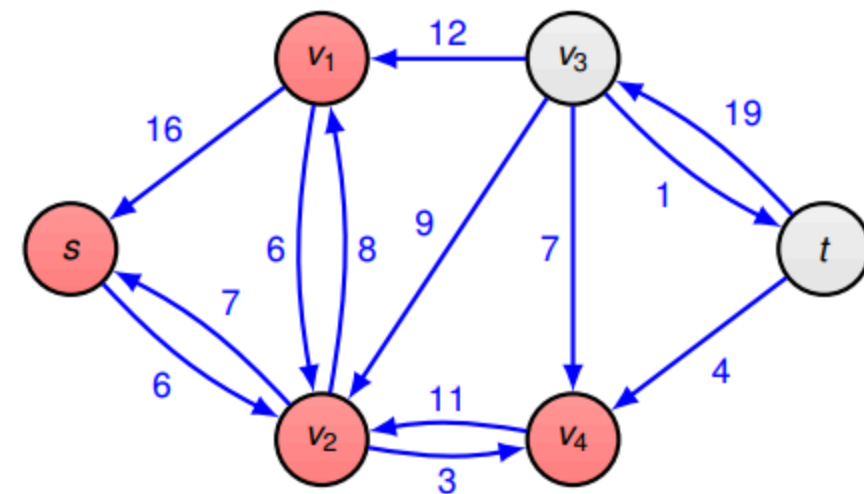
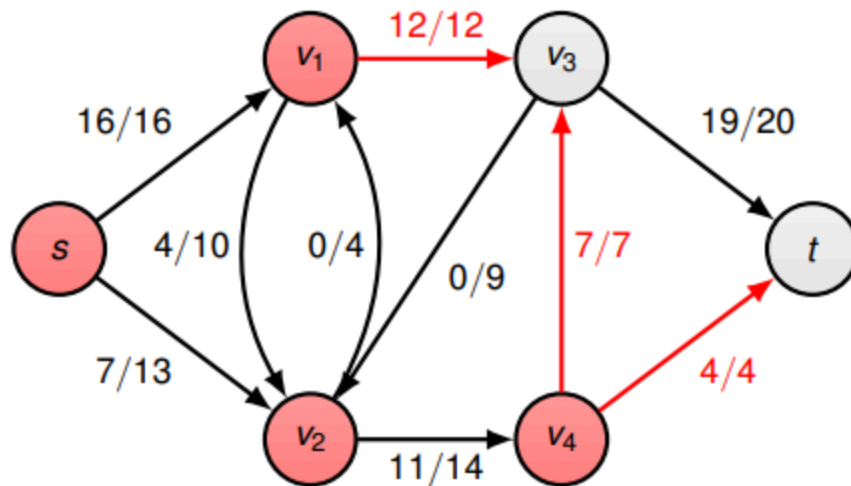
Dado un corte en una red su valor es la suma de las capacidades de todas las aristas que van de un nodo de U a uno de V .





Teorema de Max-Flow Min-Cut

- El valor del corte mínimo coincide con el del flujo máximo.
- Podemos encontrar un corte mínimo definiendo:
 - U como los nodos alcanzables desde s en la red residual del flujo máximo.
 - V como el resto de los nodos.





Aplicaciones del corte mínimo

Más allá del transporte de agua o tráfico, el corte mínimo permite resolver problemas de optimización combinatoria que, a primera vista, no parecen tener relación con redes de flujo.

La clave está en **modelar** el problema como una elección binaria por cada elemento, con penalizaciones asociadas a cada posible valor.



El problema de selección de proyectos

Project Selection Problem: se tienen n proyectos y m máquinas. Cada proyecto i genera una ganancia p_i , y cada máquina j tiene un costo q_j . Para ejecutar un proyecto, es obligatorio haber comprado todas las máquinas que requiere.

Objetivo: elegir qué proyectos ejecutar y qué máquinas comprar para **maximizar la ganancia neta**.

Se construye una red donde cada proyecto y cada máquina es un nodo:

- Arista **fuente** $\rightarrow$ **proyecto** i con capacidad p_i (costo de no tomar el proyecto)
- Arista **máquina** $j \rightarrow$ **sumidero** con capacidad q_j (costo de no comprar la máquina)
- Arista **proyecto** $i \rightarrow$ **máquina** j con capacidad ∞ , si el proyecto i necesita la máquina j

El **corte mínimo** de esta red da directamente la solución óptima al problema.



¿Por qué funciona?

La arista de capacidad ∞ entre proyecto y máquina obliga a que, si se decide tomar el proyecto, **también se debe "cortar" la compra de la máquina** de lo contrario el corte no sería válido (quedaría un camino de s a t).

Esto captura exactamente la restricción de dependencia del problema original, usando solo la estructura del corte mínimo.



Una técnica general

Este mismo patrón — variable binaria por elemento, penalización por elegir cada valor, penalización infinita para combinaciones prohibidas aparece en muchos problemas de programación competitiva:

- Selección de celdas en una grilla con restricciones de vecindad
- Problemas de coloreo con penalizaciones por adyacencia
- Restricciones de tipo "si tomas A, no puedes evitar B"

Referencia: errorgorn, "Solving Problems with Min Cut Max Flow Duality", Codeforces (blog 136761).



Problemas de Min-Cut

- CSES - Download Speed
- CSES - Police Chase
- CF 1082G - Petya and Graph

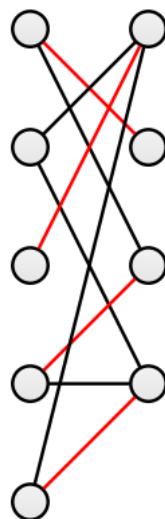


Matching máximo

Dado un grafo G , un matching es un conjunto de aristas tal que no haya dos que incidan sobre el mismo vértice.

El problema de matching máximo pide el mayor conjunto de aristas que cumpla esto.

Esta versión general del problema se puede resolver con el algoritmo de **Blossom**.





Matching máximo bipartito

Si el grafo es bipartito se puede modelar con un flujo de la siguiente manera:

- Si A y B son los conjuntos de nodos de cada lado.
- Creo un nodo s y lo conecto con capacidad 1 a cada nodo de A .
- Creo un nodo t y conecto cada nodo de B a t con capacidad 1.
- Por cada arista $a \rightarrow b$ del grafo uno a con b con capacidad ∞ .

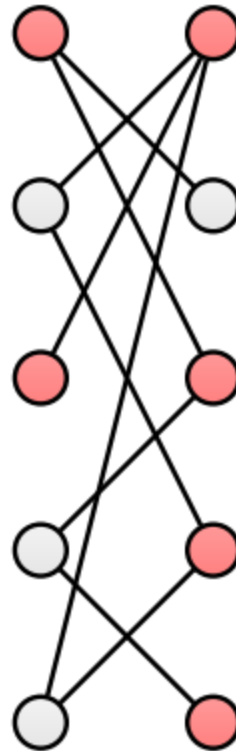
El flujo máximo sobre esta red nos da el matching máximo.

Las aristas del medio por las que enviamos flujo forman el matching.



Minimal Vertex Cover

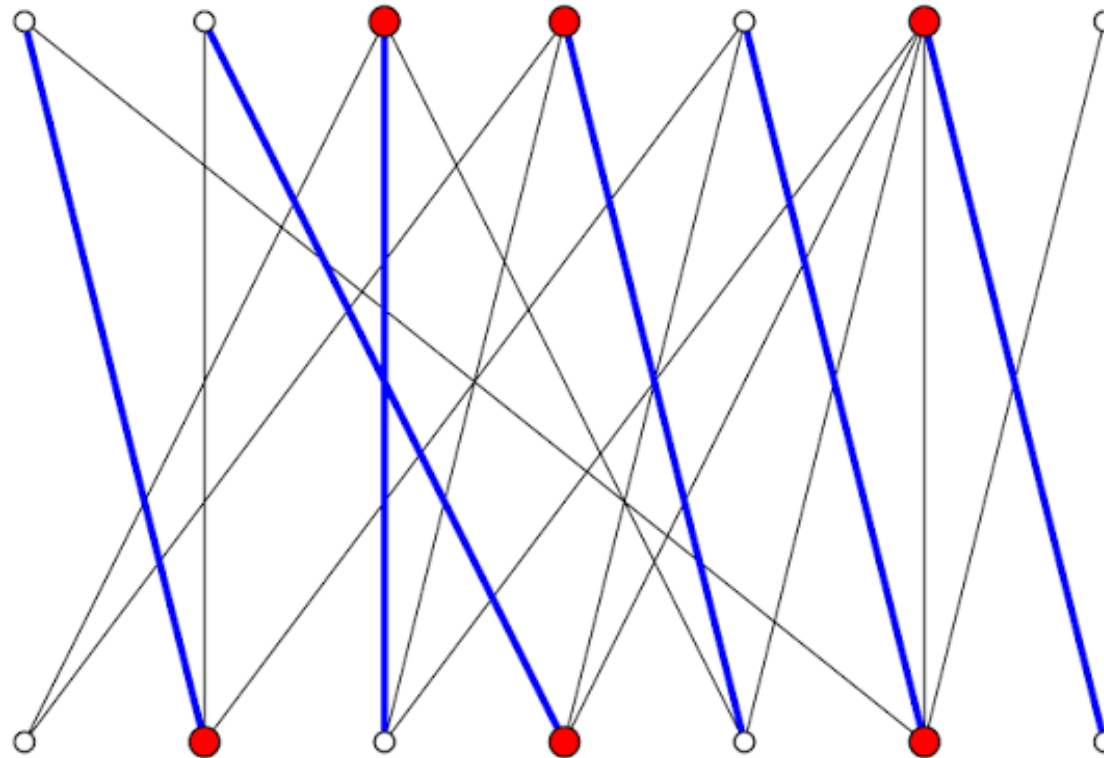
Dado un grafo, un cubrimiento por nodos es un conjunto de nodos tal que toda arista tenga algún extremo en el conjunto. Nuevamente nos interesa el cubrimiento de menor tamaño.





Minimal Vertex Cover caso bipartito

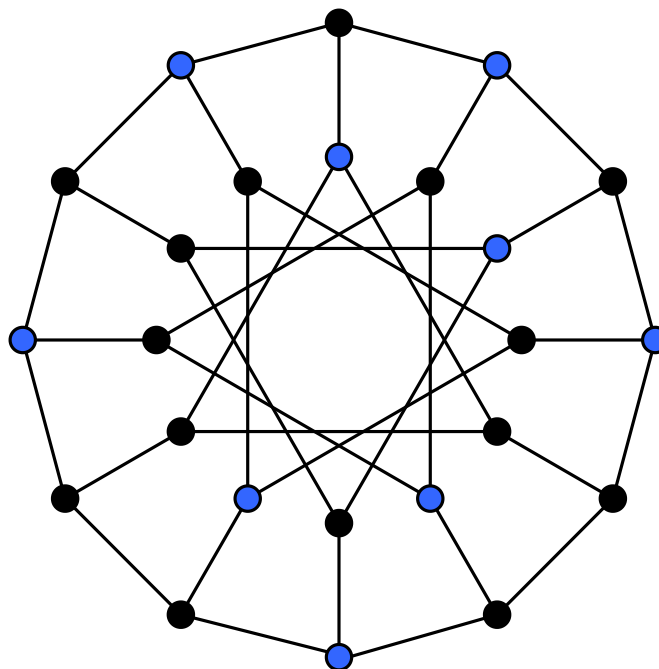
Si el grafo es bipartito, el **teorema de König** nos dice que el tamaño del cubrimiento mínimo coincide con el del matching máximo.





Maximum Independent Set

Dado un grafo, un conjunto independiente es un conjunto de nodos tal que no hay dos adyacentes. Queremos encontrar el conjunto independiente más grande.





Maximum Independent Set caso bipartito

Si el grafo es bipartito:

- Tomo un Vertex Cover mínimo.
- Los nodos que no están en el cubrimiento forman un conjunto independiente máximo.

Nuevamente, el tamaño del conjunto independiente más grande es $|V| - k$.



Problemas de Maximum Independent Set y Vertex Cover

- CF 2026E - Best subsequence
- CSES 1706 - School Dance



Maximum closure

Un **maximum closure** de un grafo dirigido es un conjunto de vértices C tal que ninguna arista sale de C hacia afuera: si un nodo está en C , todos los nodos a los que apunta también deben estar en C .

Cada vértice tiene un **peso** (positivo o negativo), y el objetivo es encontrar el closure cuya suma de pesos sea **máxima**.



Modelando el maximum closure como flujo

Se construye un grafo auxiliar agregando una fuente s y un sumidero t :

- Por cada vértice v con peso **positivo**, se agrega una arista $s \rightarrow v$ con capacidad igual a su peso
- Por cada vértice v con peso **negativo**, se agrega una arista $v \rightarrow t$ con capacidad igual al valor absoluto de su peso
- Todas las aristas



Otros problemas de flujo

Una variante del problema de flujo máximo es la siguiente:

- A cada arista, le agregamos un costo de mandar una unidad de flujo por esa arista.
- El costo de un flujo es la suma de los costos de las aristas multiplicado por el flujo que mandamos por cada una.
- Queremos encontrar entre todos los flujos máximos, aquel que minimice el costo total.



Algoritmo para encontrar el Min-Cost Max-Flow

```
template<class T1, class T2>
struct min_cost_flow {
    struct edge {int v; T1 c,f; T2 w; };
    vector<vector<int>> g; vector<T2> dist,pot; vector<T1> fl;
    vector<bool> vis; vector<int> par; vector<edge> e;
    min_cost_flow(int n):g(n),dist(n),pot(n),fl(n),vis(n),par(n){}
    void add_edge(int u, int v, T1 c, T2 w){
        int k = e.size();
        e.push_back({v,c,0,w}); e.push_back({u,c,c,-w});
        g[u].push_back(k); g[v].push_back(k^1);
    }
    using pli = pair<T2,int>;
    void dijk(int s){
        fill(dist.begin(),dist.end(),numeric_limits<T2>::max());
        fill(vis.begin(),vis.end(),0); fl[s] = numeric_limits<T1>::max();
        priority_queue<pli,vector<pli>,greater<pli>> q;
        q.push({0,s}); dist[s] = 0;
        while (!q.empty()){
            pli p = q.top(); q.pop(); int x = p.second;
            if (vis[x]) continue; vis[x] = 1;
            for (int to : g[x]){
                T2 d2 = p.first+e[to].w+pot[x]-pot[e[to].v];
                if (!vis[e[to].v] && e[to].f < e[to].c && d2 < dist[e[to].v]){
                    dist[e[to].v] = d2; fl[e[to].v] = min(fl[x],e[to].c-e[to].f);
                    par[e[to].v] = to; q.push({d2,e[to].v});
                }
            }
        }
    }
    T2 calc_flow(int s, int t, T1 _f){
        T2 cost = 0;
        while (1){
            dijk(s);
            if (!vis[t]) return -1;
            for (int i = 0; i < g.size(); i++)
                dist[i] += pot[i]-pot[s];
            T1 mnf = min(_f,fl[t]);
            cost += dist[t]*mnf;
            int cur = t;
            while (cur != s){
                e[par[cur]].f += mnf;
                e[par[cur]^1].f -= mnf;
                cur = e[par[cur]^1].v;
            }
            dist.swap(pot);
            _f -= mnf;
            if (!_f) break;
        }
        return cost;
    }
};
```



En caso de que existan costos negativos en las aristas, no se puede usar Dijkstra directamente para encontrar el camino más corto en cada iteración del algoritmo de flujo de costo mínimo, ya que Dijkstra asume pesos no negativos. En su lugar, hay que reemplazarlo por el algoritmo de **Bellman-Ford** (o su variante optimizada, SPFA), que sí admite aristas de costo negativo.

Cuando todos los costos son no negativos, se puede usar **Dijkstra** en cada iteración, con una complejidad total de $O(F \cdot E \log V)$, donde F es el valor del flujo máximo.

En cambio, si hay costos negativos y se recurre a Bellman-Ford en cada iteración, la complejidad sube a $O(F \cdot V \cdot E)$ considerablemente más lenta, ya que Bellman-Ford es asintóticamente más costoso que Dijkstra por cada búsqueda de camino más corto.



¿Y si nos preguntan Min-Cost Max-Flow para un matching máximo?

- Podemos modelar el problema exactamente igual y utilizamos el algoritmo de Min-Cost Max-Flow en el grafo
- (Opción más rápida) Utilizar el **Hungarian algorithm** que lo resuelve en $O(n^3)$



Hungarian algorithm

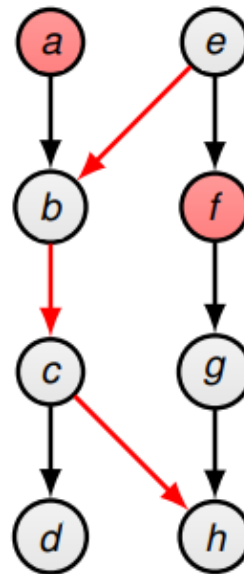
```
void Hungarian(vector<vector<ll>> &A, vector<pair<int, int>> &result, ll &C, const ll INF = 1e15) {
    int n = A.size() - 1, m = A[0].size() - 1;
    vector<ll> minv(m + 1), u(n + 1), v(m + 1);
    vector<int> p(m + 1), way(m + 1);
    vector<bool> used(m + 1);
    for (int i = 1; i <= n; ++i) {
        p[0] = i; int j0 = 0;
        for (int j = 0; j <= m; ++j)
            minv[j] = INF;
        for (int j = 0; j <= m; ++j)
            used[j] = false;
        do {
            used[j0] = true;
            int i0 = p[j0], j1;
            ll delta = INF;
            for (int j = 1; j <= m; ++j)
                if (!used[j]) {
                    ll cur = A[i0][j] - u[i0] - v[j];
                    if (cur < minv[j]) minv[j] = cur, way[j] = j0;
                    if (minv[j] < delta) delta = minv[j], j1 = j;
                }
            for (int j = 0; j <= m; ++j) {
                if (used[j]) u[p[j]] += delta, v[j] -= delta;
                else minv[j] -= delta;
            }
            j0 = j1;
        } while (p[j0] != 0);
        do {
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        } while(j0);
    }
    for (int i = 1; i <= m; ++i)
        result.push_back(make_pair(p[i], i));
    C = -v[0];
}
```

- CF 237E - Build String
- CSES 2130 - Distinct Routes II



Dado un grafo acíclico dirigido (DAG) tenemos las siguientes definiciones:

- Una **cadena** es una secuencia de nodos tal que de cada uno puedo llegar al siguiente.
- Una **anticadena** es un conjunto de nodos tal que no hay camino entre ningún par de ellos.





Teorema de Dilworth

El teorema de Dilworth nos dice que los tamaños de estas cosas coinciden:

- La anticadena de mayor tamaño
- La menor cantidad de cadenas que cubren todos los nodos.



Construimos el siguiente grafo bipartito:

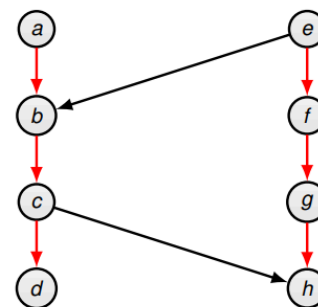
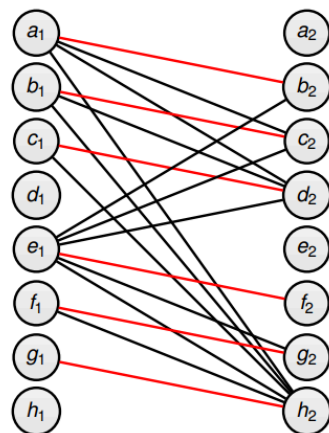
- Ponemos una copia de cada nodo del grafo de cada lado del bipartito.
- Unimos un nodo del lado izquierdo u con uno del derecho v si hay camino de u a v en el grafo original.



Minimal Chain Cover

Para el cubrimiento por cadenas:

- Obtenemos un matching máximo del grafo.
- Si un eje de u a v está en el matching, agregamos la cadena formada por el camino de u a v en el grafo.
- Si las dos copias de un nodo están en el matching, unimos las dos cadenas con extremos en este nodo.

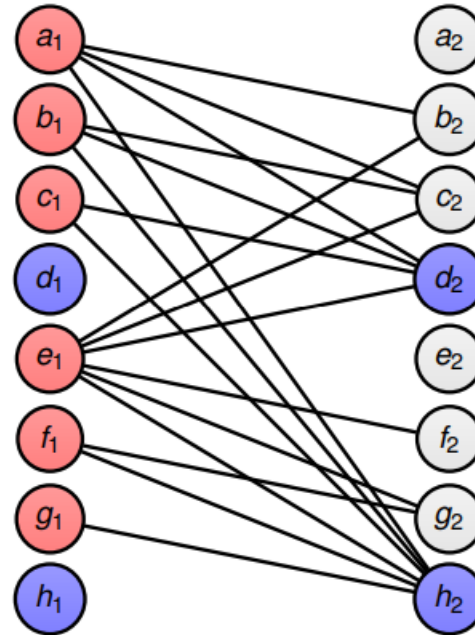




Anticadena máxima

Para la máxima anticadena:

- Obtenemos un Minimal Vertex Cover del grafo.
- Tomamos los nodos que no tienen ninguna copia en el cubrimiento y este es el máximo tamaño de la anticadena.





Problemas de Dilworth

- [UVA 11159 - Factors and Multiples](#)
- [SPOJ DIVREL - Divisible Relation](#)